

Unser Thema ist Next Level

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für Informatik

Eingereicht von:

Stefan Schwammal
Stefanie Schwammal

Betreuer:

Lukas Lehrer

Projektpartner:

Petra Programmierer, Initech Inc.

Leonding, 4. April 2025

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 4. April 2025

Stefan Schwammal & Stefanie Schwammal

Abstract

Brief summary of our amazing work. In English. This is the only time we have to include a picture within the text. The picture should somehow represent your thesis. This is untypical for scientific work but required by the powers that are. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.



Zusammenfassung

Zusammenfassung unserer genialen Arbeit. Auf Deutsch. Das ist das einzige Mal, dass eine Grafik in den Textfluss eingebunden wird. Die gewählte Grafik soll irgendwie eure Arbeit repräsentieren. Das ist ungewöhnlich für eine wissenschaftliche Arbeit aber eine Anforderung der Obrigkeit. *Bitte auf keinen Fall mit der Zusammenfassung verwechseln, die den Abschluss der Arbeit bildet!* Suspendisse vel felis.

Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.



Inhaltsverzeichnis

1	Einleitung	1
2	Quarkus	2
2.1	Quarkus – Grundlagen und Einsatz in der Anwendung	2
2.2	Grundprinzip von Quarkus	2
2.3	Quarkus in der vorliegenden Applikation	4
2.4	Backend-Architektur	4
2.5	Implementierung des Backends in der vorliegenden Applikation	7
2.6	Datenbankkonzept und architektonische Entscheidungen	12
3	Architektur	17
3.1	Datenbanken	17
3.2	Datenbankkonzept und architektonische Entscheidungen	18
4	Datenbank	23
4.1	Datenmodell für Abonnements und Medien	23
4.2	Begründung für relationale Datenhaltung	23
5	Umsetzung	24
6	Zusammenfassung	26
	Glossar	V
	Literaturverzeichnis	VI
	Abbildungsverzeichnis	VII
	Tabellenverzeichnis	VIII
	Quellcodeverzeichnis	IX

1 Einleitung

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

2 Quarkus

2.1 Quarkus – Grundlagen und Einsatz in der Anwendung

Quarkus ist ein modernes, quelloffenes Java-Framework, das speziell für den Einsatz in containerisierten und Cloud-nativen Umgebungen entwickelt wurde. Ziel des Frameworks ist es, den Prozess der Bereitstellung von Java-Applikationen in Cloud-Umgebungen zu vereinfachen und zu optimieren. Dabei unterstützt Quarkus Entwicklerinnen und Entwickler dabei, Applikationen auch ohne tiefgehende Kenntnisse der zugrundeliegenden Infrastruktur bereitzustellen.

Ein wesentliches Merkmal von Quarkus ist seine Kompatibilität mit einer Vielzahl etablierter Frameworks und Standards. Dazu zählen unter anderem Eclipse MicroProfile, RESTEasy (JAX-RS), Hibernate ORM (JPA), Apache Kafka sowie Spring. Diese breite Unterstützung ermöglicht es, bestehende Kenntnisse und Bibliotheken direkt einzusetzen, ohne grundlegende Umstrukturierungen vornehmen zu müssen.

Quarkus hebt sich durch seine Leichtgewichtigkeit von anderen Java-Frameworks ab. Konkret äußert sich dies in einer geringen Startzeit, einem niedrigen Speicherverbrauch sowie der einfachen Möglichkeit, Projekte in Cloud-Umgebungen bereitzustellen.

2.2 Grundprinzip von Quarkus

Ein zentrales Merkmal von Quarkus liegt in der Art, wie Java-Anwendungen kompiliert und ausgeführt werden. Um dies zu verstehen, ist es notwendig, die zwei grundlegenden Kompilierungsansätze zu unterscheiden: die Just-In-Time-Kompilierung (JIT) und die Ahead-of-Time-Kompilierung (AOT).

2.2.1 Just-In-Time-Kompilierung (JIT)

Im klassischen Java-Ausführungsmodell wird Quellcode zunächst vom Java-Compiler in plattformunabhängigen Bytecode übersetzt. Dieser Bytecode wird anschließend von der Java Virtual Machine (JVM) interpretiert. Die JIT-Kompilierung ist dabei ein Optimierungsmechanismus der JVM: Häufig ausgeführte Codeabschnitte werden zur Laufzeit in nativen Maschinencode übersetzt, was die Ausführungsgeschwindigkeit erhöht.

Der Nachteil dieses Ansatzes liegt darin, dass die JVM beim Start der Applikation zunächst hochgefahren werden muss und die JIT-Optimierungen erst schrittweise während der Laufzeit greifen. Dies führt zu längeren Startzeiten und einem erhöhten Speicherverbrauch – Faktoren, die in containerisierten Umgebungen mit häufigen Neustarts besonders nachteilig sind.

2.2.2 Ahead-of-Time-Kompilierung (AOT)

Die Ahead-of-Time-Kompilierung verfolgt einen grundlegend anderen Ansatz: Der gesamte Anwendungscode wird bereits vor der Ausführung, also zum Zeitpunkt des Build-Prozesses, in nativen Maschinencode übersetzt. Quarkus nutzt hierfür GraalVM Native Image, ein Werkzeug, das Java-Anwendungen in plattformspezifische, ausführbare Binärdateien kompiliert.

Das Ergebnis ist eine native ausführbare Datei, die ohne eine laufende JVM auskommt. Dies hat folgende Vorteile:

- **Schnelle Startzeit:** Da keine JVM initialisiert werden muss und alle Optimierungen bereits zur Build-Zeit vorgenommen wurden, startet die Anwendung in Millisekunden.
- **Geringer Speicherverbrauch:** Der Wegfall der JVM-Laufzeitumgebung reduziert den RAM-Bedarf erheblich.
- **Eignung für containerisierte Umgebungen:** Kurze Startzeiten und geringer Ressourcenverbrauch machen AOT-kompilierte Anwendungen besonders geeignet für Kubernetes und ähnliche Plattformen.

Als Nachteil ist die deutlich längere Build-Zeit im Vergleich zur JIT-Kompilierung zu nennen, da die gesamte Kompilierung und Optimierung vorab stattfindet.

2.2.3 Hot Reload

Für den Entwicklungsprozess bietet Quarkus einen sogenannten Hot-Reload-Mechanismus, bekannt als *Quarkus Dev Mode*. Dabei werden Änderungen am Quellcode automatisch erkannt und ohne Neustart der Anwendung übernommen. Dies beschleunigt den Entwicklungszyklus erheblich, da zeitaufwändige Neustarts entfallen. Der Dev Mode verwendet intern JIT-Kompilierung, um diese schnelle Rückkopplungsschleife zu ermöglichen – die AOT-Kompilierung wird lediglich für den finalen Build eingesetzt.

2.3 Quarkus in der vorliegenden Applikation

Quarkus wird in der vorliegenden Anwendung als Hauptframework für das Backend verwendet. Für die Datenbankbindung kommt Hibernate ORM mit Panache zum Einsatz, das eine vereinfachte Abstraktion des JPA-Standards bietet und den Datenbankzugriff mit weniger Boilerplate-Code ermöglicht.

Die Anwendung wird mittels AOT-Kompilierung als native ausführbare Datei gebaut. Obwohl die Build-Zeit dadurch im Vergleich zur klassischen JIT-Kompilierung länger ausfällt, überwiegen die Vorteile im Betrieb: Die Applikation weist eine deutlich verkürzte Startzeit auf, benötigt weniger Arbeitsspeicher und ist optimal für den Einsatz in einer containerisierten Umgebung geeignet. Diese Eigenschaften sind insbesondere in Cloud-Deployments von Vorteil, wo Ressourceneffizienz und Skalierbarkeit zentrale Anforderungen darstellen.

2.4 Backend-Architektur

Die Backend-Architektur der vorliegenden Anwendung folgt einem mehrschichtigen Aufbau, der eine klare Trennung der Verantwortlichkeiten gewährleistet. Dieses Architekturmuster wird auch als *Layered Architecture* bezeichnet. Jede Schicht übernimmt eine klar definierte Aufgabe, wodurch die Wartbarkeit, Testbarkeit und Erweiterbarkeit der Anwendung verbessert wird. Eine eingehende HTTP-Anfrage durchläuft dabei die Resource-Schicht, wird über DTOs weitergeleitet, von der Service-Schicht verarbeitet und schließlich über das Repository auf die Datenbankentitäten abgebildet.

2.4.1 Entity

Die Entity-Schicht repräsentiert die Datenmodelle einer Anwendung. Jede Entity-Klasse bildet die Struktur einer Datenbanktabelle ab und wird mithilfe von Annotationen der Jakarta Persistence API (JPA) mit der Datenbank verknüpft. Entity-Klassen enthalten ausschließlich die Datenbankstruktur sowie einfache Zugriffsmethoden. Geschäftslogik hat in dieser Schicht keine Berechtigung, da dies die Trennung der Verantwortlichkeiten verletzen würde.

2.4.2 Repository

Die Repository-Schicht kapselt den Zugriff auf die Datenbank und stellt eine Abstraktion zwischen der Datenzugriffslogik und den darüberliegenden Schichten dar. Sie definiert die grundlegenden CRUD-Operationen (Create, Read, Update, Delete) sowie komplexere Datenbankabfragen. Durch diese Trennung bleibt die Geschäftslogik unabhängig von der konkreten Datenbankimplementierung, was einen späteren Austausch der Persistenzschicht erleichtert und die Testbarkeit der Anwendung erhöht.

2.4.3 Service

Die Service-Schicht beinhaltet die Geschäftslogik der Anwendung und fungiert als Vermittler zwischen der Resource- und der Repository-Schicht. Sie ist verantwortlich für die Verarbeitung und Transformation von Daten, die Koordination mehrerer Repository-Zugriffe sowie die Kommunikation mit externen Systemen. Die Service-Schicht stellt sicher, dass die Geschäftsregeln der Anwendung zentral und unabhängig von der Präsentations- oder Datenzugriffsschicht definiert sind.

2.4.4 Resource

Die Resource-Schicht ist für die Bereitstellung der RESTful-Endpunkte verantwortlich. Sie nimmt eingehende HTTP-Anfragen entgegen, delegiert die Verarbeitung an die zuständigen Service-Klassen und gibt die entsprechenden Antworten an den Client zurück. Die Resource-Schicht enthält dabei selbst keine Geschäftslogik, sondern fungiert ausschließlich als Schnittstelle zwischen dem Backend und den konsumierenden Clients. Durch diese strikte Trennung bleibt die Schicht schlank und leicht austauschbar.

2.4.5 DTO – Data Transfer Objects

Data Transfer Objects (DTOs) sind speziell zugeschnittene Datenobjekte, die dem gezielten Datenaustausch zwischen verschiedenen Schichten oder zwischen Backend und Client dienen. Anstatt vollständige Entity-Objekte oder rohe API-Antworten direkt weiterzugeben, werden DTOs verwendet, die ausschließlich jene Felder enthalten, die für den jeweiligen Anwendungsfall erforderlich sind.

DTOs erfüllen dabei zwei wesentliche Zwecke. Erstens erhöhen sie die Sicherheit der Anwendung: Sensible Felder einer Entity – wie etwa Passwort-Hashes oder interne Systemfelder – werden nicht versehentlich nach außen weitergegeben. Zweitens wird eine Entkopplung zwischen der internen Datenstruktur und der nach außen sichtbaren API-Struktur erreicht. Änderungen am Datenbankmodell oder an einer externen API haben dadurch keinen direkten Einfluss auf die eigene Schnittstelle.

Motivation am konkreten Beispiel: TMDB API

Um den praktischen Nutzen eines DTOs zu veranschaulichen, eignet sich der Vergleich zwischen der rohen Antwort der TMDB API und dem in der vorliegenden Anwendung verwendeten `TrendingMovieDTO`.

Ein einzelnes Filmobjekt aus der TMDB-Antwort des Endpunkts `/trending/movie/{timewindow}` enthält unter anderem folgende Felder:

Listing 1: Auszug einer TMDB API-Antwort für einen Film

```
1  {
2    "backdrop_path": "/avedvodAZUcwqevBfm8p4G2NziQ.jpg",
3    "id": 933260,
4    "title": "The Substance",
5    "original_title": "The Substance",
6    "overview": "A fading celebrity ...",
7    "poster_path": "/lqoMzCcZYEfK729d6qzt349fB4o.jpg",
8    "media_type": "movie",
9    "adult": false,
10   "original_language": "en",
11   "genre_ids": [27, 878],
12   "popularity": 4743.21,
13   "release_date": "2024-09-07",
14   "video": false,
15   "vote_average": 7.1,
16   "vote_count": 3890
17 }
```

Die TMDb API liefert pro Film 14 Felder zurück, von denen für die vorliegende Anwendung jedoch nur ein Teil relevant ist. Felder wie `backdrop_path`, `adult`, `original_language`, `genre_ids`, `media_type` oder `video` werden in der Anwendung nicht benötigt und würden bei einer ungefilterten Weitergabe an den Client unnötig Bandbreite verbrauchen und die Antwortstruktur verkomplizieren.

Stattdessen wird die Rohantwort im `TmdbService` verarbeitet und auf den `TrendingMovieDTO` abgebildet, der ausschließlich die relevanten Felder enthält:

Listing 2: `TrendingMovieDTO` als Java Record

```
1 public record TrendingMovieDTO(  
2     Long id,  
3     String title,  
4     String poster,  
5     String overview,  
6     double popularity,  
7     LocalDate releaseDate,  
8     double voteAverage,  
9     double voteCount  
10 ) {}
```

Der resultierende DTO enthält lediglich 8 statt 14 Felder. Darüber hinaus wird das Feld `poster_path` – das in der TMDb-Antwort nur einen relativen Pfad wie `/1qoMzCcZYEFK729d6qzt349fB4o.jpg` enthält – bereits im Service zu einer vollständigen URL zusammengesetzt:

Listing 3: Zusammensetzung der Poster-URL im `TmdbService`

```
1 String poster = "https://image.tmdb.org/t/p/w92 "  
2     + movie.path("poster_path").asText(null);
```

Der Client erhält damit eine direkt verwendbare URL, ohne selbst Kenntnisse über die Bildpfad-Konvention der TMDb API besitzen zu müssen. Dies ist ein weiterer Vorteil des DTO-Patterns: Die Komplexität der externen API wird im Backend gekapselt und dem Client eine saubere, konsistente Schnittstelle präsentiert.

2.5 Implementierung des Backends in der vorliegenden Applikation

Die im vorherigen Abschnitt beschriebenen Architekturprinzipien wurden in der vorliegenden Anwendung konsequent umgesetzt. Das Backend basiert auf Quarkus und

verwendet Hibernate ORM für die Datenbankbindung. Die folgende Beschreibung erläutert, wie die einzelnen Schichten konkret implementiert wurden.

2.5.1 Entity

Die Datenbankstruktur der Anwendung wird durch fünf Entity-Klassen sowie ein Enum abgebildet, die gemeinsam das Domänenmodell der Anwendung repräsentieren.

Die zentrale Entität ist **UserMovieDB**, welche einen registrierten Benutzer der Anwendung darstellt. Als Primärschlüssel wird dabei keine automatisch generierte Datenbank-ID verwendet, sondern die **UUID** des Benutzers aus dem Identity-Provider Keycloak. Dies stellt sicher, dass die interne Benutzerrepräsentation direkt mit dem Authentifizierungssystem verknüpft ist.

Die Entität **Provider** repräsentiert einen Streaming-Anbieter wie etwa Netflix oder Amazon Prime. Neben einer internen Datenbank-ID speichert sie die **tmdbProviderId**, also die eindeutige ID des Anbieters in der TMDb API. Diese dient als Brücke zwischen den intern gespeicherten Providern und den von der externen API gelieferten Daten. Jedem Provider kann zusätzlich eine Liste von **CountryPriority**-Einträgen zugeordnet werden, die die Anzeigepriorität des Anbieters je nach Land abbilden.

Die Verknüpfung zwischen Benutzer und Streaming-Anbieter erfolgt über die Entität **UserProviderSubscription**. Diese speichert neben den Fremdschlüsseln auf **UserMovieDB** und **Provider** auch den Abrechnungszyklus des Abonnements sowie das letzte Abrechnungsdatum. Der Abrechnungszyklus wird über das Enum **BillingCycle** mit den Ausprägungen **MONTHLY** und **YEARLY** abgebildet. Eine eindeutige Kombination aus Benutzer und Anbieter wird durch eine **UniqueConstraint**-Annotation auf Datenbankebene sichergestellt, sodass ein Benutzer denselben Anbieter nicht mehrfach abonnieren kann. Die Entität stellt zudem eine berechnete Methode **getNextBillingDate()** bereit, die basierend auf dem Abrechnungszyklus das nächste Fälligkeitsdatum ermittelt.

Die Entität **History** speichert den Filmschauverlauf eines Benutzers. Sie referenziert den jeweiligen Benutzer über einen Fremdschlüssel sowie die **tmdbMovieId** des gesehenen Films. Der Zeitpunkt des Ansehens wird automatisch beim Erstellen eines **History**-Eintrags auf den aktuellen Zeitstempel gesetzt.

2.5.2 Repository

Die Repository-Schicht der Anwendung besteht aus drei Klassen, die jeweils den Datenbankzugriff für eine bestimmte Entität kapseln.

Das `UserMovieDBRepository` implementiert das `PanacheRepository`-Interface und stellt spezialisierte Abfragemethoden bereit. Die Methode `findByUUID` sucht einen Benutzer anhand seiner Keycloak-UUID und gibt ein `Optional` zurück, um den Fall eines nicht vorhandenen Benutzers typischer zu behandeln. Die Methode `getUserProviderTmdbIds` liefert alle TMDB-Provider-IDs der Abonnements eines bestimmten Benutzers als `Set`, das anschließend im Service-Layer für den Abgleich mit der TMDB API verwendet wird. Besonders hervorzuheben ist die Methode `getProvidersWithOwnership`, die mithilfe einer JPQL-Abfrage alle Provider aus der Datenbank lädt und dabei über einen `LEFT JOIN` mit den Abonnements des Benutzers verknüpft. Das Ergebnis enthält für jeden Provider ein Flag, das angibt, ob der Benutzer diesen abonniert hat. Dadurch kann das Frontend mit einem einzigen Request alle relevanten Informationen abrufen. Darüber hinaus enthält das Repository die Methode `updateSubscriptions`, die die Abonnements eines Benutzers anhand einer übergebenen Liste von `SubscriptionUpdatedDTO`-Objekten aktualisiert. Dabei werden neue Abonnements angelegt, bestehende aktualisiert und nicht mehr enthaltene entfernt.

Das `ProviderRepository` verwaltet die Streaming-Anbieter in der Datenbank. Die zentrale Methode ist `upsertFromTmdb`, die beim Anwendungsstart aufgerufen wird: Sie prüft, ob ein Provider mit der gegebenen `tmdbProviderId` bereits existiert, und aktualisiert dessen Daten oder legt einen neuen Eintrag an. Dadurch wird eine konsistente Synchronisierung der Provider-Daten mit der TMDB API gewährleistet.

Das `HistoryRepository` implementiert ebenfalls das `PanacheRepository`-Interface und stellt Methoden für den Zugriff auf den Filmschauerverlauf bereit. Die Methode `findByIdUserId` gibt alle History-Einträge eines Benutzers sortiert nach dem Zeitpunkt des Ansehens zurück. Über `findByIdUserIdLimit` kann die Anzahl der zurückgegebenen Einträge begrenzt werden, was eine einfache Paginierung ermöglicht. Die Methode `hasWatchedMovie` prüft, ob ein Benutzer einen bestimmten Film bereits gesehen hat.

2.5.3 Service

Die Service-Schicht der Anwendung besteht aus drei Klassen, die jeweils einen klar abgegrenzten Verantwortungsbereich abdecken.

Der `UserService` ist für die Verwaltung des aktuell authentifizierten Benutzers zuständig. Er liest die Keycloak-UUID aus dem JWT-Token aus und prüft über das `UserMovieDBRepository`, ob der Benutzer bereits in der Datenbank existiert. Ist dies nicht der Fall, wird automatisch ein neuer `UserMovieDB`-Eintrag angelegt. Dieses Muster wird als *Lazy User Creation* bezeichnet und stellt sicher, dass beim ersten authentifizierten Zugriff eines neuen Benutzers automatisch ein Datenbankprofil erstellt wird, ohne dass eine explizite Registrierung erforderlich ist.

Der `TmdbService` kapselt die gesamte Kommunikation mit der externen TMDB API. Die HTTP-Anfragen werden über die Bibliothek `OkHttp` durchgeführt, die empfangenen JSON-Antworten mit Jackson geparkt und auf DTOs abgebildet. Der API-Schlüssel wird nicht im Quellcode hinterlegt, sondern über den Quarkus-Konfigurationsmechanismus mittels `@ConfigProperty` injiziert, was die Sicherheit der Anwendung erhöht. Der Service stellt Methoden zum Abrufen von Filminformationen per ID oder Suchbegriff, zum Laden verfügbarer Streaming-Anbieter sowie zum Abrufen trendender Filme bereit. Die Methode `getFilteredProvidersForUser` kombiniert dabei externe API-Daten mit internen Datenbankdaten: Sie ruft die verfügbaren Provider eines Films von der TMDB API ab, gleicht diese mit den vom Benutzer abonnierten Anbietern aus der Datenbank ab und setzt das Flag `ownedByUser` im zurückgegebenen `ProviderInfoDTO`. Die Ergebnisliste wird abschließend so sortiert, dass die vom Benutzer abonnierten Anbieter an erster Stelle erscheinen.

Der `ProviderSyncService` wird beim Start der Anwendung automatisch ausgeführt, indem er auf das Quarkus-Lifecycle-Event `StartupEvent` reagiert. Er ruft über den `TmdbService` alle verfügbaren Streaming-Anbieter von der TMDB API ab und synchronisiert diese über die `upsertFromTmdb`-Methode des `ProviderRepository` in die lokale Datenbank. Dadurch ist sichergestellt, dass die Anwendung stets eine aktuelle Liste aller Provider vorhält, ohne dass ein manueller Synchronisierungsschritt erforderlich ist. Der gesamte Vorgang wird in einer Transaktion ausgeführt, was durch die `@Transactional`-Annotation sichergestellt wird.

2.5.4 Resource

Die Resource-Schicht der Anwendung besteht aus drei produktiven Resource-Klassen, die die REST-Endpunkte des Backends definieren.

Die `MovieResource` stellt Endpunkte unter dem Basispfad `/api/movies` bereit. Sie ermöglicht das Abrufen von Filmdaten anhand einer ID oder eines Suchbegriffs, das Laden verfügbarer Streaming-Anbieter für einen Film sowie das Abrufen trendender Filme. Einige Endpunkte sind öffentlich zugänglich und mit `@PermitAll` annotiert, während der Endpunkt `/api/movies/{id}/providers/for-user` eine gültige Authentifizierung erfordert und über `@RolesAllowed` auf die Rollen `user` und `medien-abo-users-role` beschränkt ist. Bei diesem Endpunkt wird die Benutzer-ID direkt aus dem JWT-Token extrahiert, sodass keine User-ID in der URL übermittelt werden muss.

Die `UserResource` stellt Endpunkte unter dem Basispfad `/api/users` bereit und ist vollständig authentifizierungsgeschützt. Sie bietet Funktionalitäten zur Verwaltung des eigenen Benutzerprofils, der Streaming-Abonnements sowie der Filmschauhistorie. Über `GET /api/users/me` kann der aktuelle Benutzer seine Profildaten abrufen, wobei bei einem erstmaligen Zugriff automatisch ein Datenbankprofil angelegt wird. Der Endpunkt `PUT /api/users/me/subscriptions` ermöglicht das vollständige Aktualisieren der Abonnements in einem einzigen Request. Die History-Endpunkte erlauben das Hinzufügen, Abrufen und Löschen von Einträgen im Filmschauverlauf. Beim Löschen wird explizit geprüft, ob der Eintrag dem authentifizierten Benutzer gehört, bevor die Operation ausgeführt wird.

Die `ProviderResource` stellt Endpunkte unter `/api/providers` bereit. Sie ermöglicht das Abrufen aller in der Datenbank gespeicherten Streaming-Anbieter sowie das manuelle Auslösen einer Synchronisierung mit der TMDB API über den Endpunkt `GET /api/providers/sync`.

2.5.5 DTO – Data Transfer Objects

In der vorliegenden Anwendung werden DTOs konsequent eingesetzt, um die Kommunikation zwischen den Schichten zu strukturieren und interne Datenbankstrukturen zu kapseln.

Der `ProviderInfoDTO` wird bei der Abfrage von Streaming-Anbietern eines Films verwendet. Er enthält die TMDB-Provider-ID, den Anbieternamen, die URL des Provider-

Logos sowie das Flag `ownedByUser`, das angibt, ob der jeweilige Benutzer diesen Anbieter abonniert hat. Die vollständige `Provider`-Entity sowie interne Datenbankfelder werden dabei nicht weitergegeben.

Der `ProviderWithOwnership`-DTO wird vom `UserMovieDBRepository` direkt über eine JPQL-Konstruktorabfrage befüllt und an den Client übermittelt. Er enthält neben den Provider-Stammdaten auch den Abrechnungszyklus und das letzte Abrechnungsdatum des Abonnements, sofern der Benutzer den jeweiligen Anbieter abonniert hat.

Der `SubscriptionUpdateDTO` dient als Eingabe-DTO für den PUT `/api/users/me/subscriptions` Endpunkt. Er enthält die TMDB-Provider-ID, den gewünschten Abrechnungszyklus sowie das Abrechnungsdatum und ermöglicht so die vollständige Aktualisierung eines Abonnements in einem einzigen Objekt.

Der `TrendingMovieDTO` bündelt alle relevanten Informationen eines trendenden Films – Titel, Poster-URL, Kurzbeschreibung, Erscheinungsdatum, Popularitätswert sowie Bewertungsdurchschnitt und Bewertungsanzahl – in einem kompakten Objekt, das direkt an den Client übermittelt wird.

Der `HistoryDTO` dient der strukturierten Rückgabe von History-Einträgen an den Client. Er wird über eine statische `from(History)`-Methode aus der gleichnamigen Entity erstellt und stellt sicher, dass keine internen JPA-Proxies oder lazy-geladenen Assoziationen versehentlich serialisiert werden.

Der `ProviderApi`-Record dient als internes Transferobjekt zwischen dem `TmdbService` und dem `ProviderSyncService`. Er bildet die Struktur der TMDB-API-Antwort ab und ermöglicht eine typsichere Weiterverarbeitung der Rohdaten, bevor diese in die Datenbank geschrieben werden.

2.6 Datenbankkonzept und architektonische Entscheidungen

Für das Backend der Anwendung spielt die Wahl der geeigneten Datenbanktechnologie eine zentrale Rolle. Trotz vorhandener Kenntnisse im Bereich nicht-relationaler Datenbanken (NoSQL) – wie etwa MongoDB oder Vektordatenbanken – wurde im Rahmen dieses Projekts bewusst eine relationale Datenbank eingesetzt.

Die Entscheidung für ein relationales Datenbanksystem basiert auf mehreren Faktoren. Die Anwendung weist eine klar strukturierte und logisch getrennte Datenarchitektur auf, wodurch sich die Daten und deren Beziehungen gut in einem relationalen Modell abbilden lassen. Entitäten und ihre Relationen konnten somit eindeutig definiert und konsistent strukturiert werden. Dieses Vorgehen entspricht dem etablierten Standard bei Anwendungen mit stark strukturierten Daten und klar definierten Geschäftsprozessen.

Ein weiterer wesentlicher Aspekt ist die Skalierbarkeit des Projekts. Relationale Datenbanken bieten durch Normalisierung, Transaktionssicherheit (ACID-Prinzip) und klar definierte Beziehungen eine stabile Grundlage für zukünftige Erweiterungen. Voraussetzung für eine nachhaltige Skalierbarkeit ist jedoch eine sorgfältig durchdachte Datenbankstruktur. Eine frühzeitig geplante und konsistente Modellierung – beispielsweise durch ein Entity-Relationship-Diagramm (ERD) – ist essenziell, um spätere strukturelle Anpassungen zu minimieren.

2.6.1 Herausforderungen im Entwicklungsprozess

Im Verlauf der Diplomarbeit traten mehrere strukturelle Herausforderungen auf, die im Sinne einer transparenten Dokumentation offen dargestellt werden sollen.

Zu Beginn der Implementierungsphase wurde ohne vollständig ausgearbeitetes Gesamtkonzept mit der Programmierung begonnen. Das Entity-Relationship-Diagramm wurde parallel zur Entwicklung erstellt, wodurch sich die Datenbankstruktur dynamisch entwickelte. Dieses iterative Vorgehen führte zwar zu schneller praktischer Umsetzung, hatte jedoch den Nachteil, dass strukturelle Änderungen im Nachhinein größeren Anpassungsaufwand verursachten.

Eine weitere wesentliche Herausforderung ergab sich durch die späte Integration von Keycloak als Identity- und Access-Management-Lösung. Ursprünglich war die Benutzerverwaltung eigenständig implementiert worden. Durch die nachträgliche Einführung von Keycloak mussten die bestehenden User-Strukturen grundlegend überarbeitet werden. Dies hatte zur Folge, dass die Benutzerentitäten angepasst werden mussten, die Authentifizierungs- und Autorisierungslogik neu strukturiert wurde sowie abhängige Klassen und Beziehungen refaktoriert und teilweise neu implementiert werden mussten. Diese Umstellung führte zu zusätzlichem Entwicklungsaufwand, zeigte jedoch zugleich

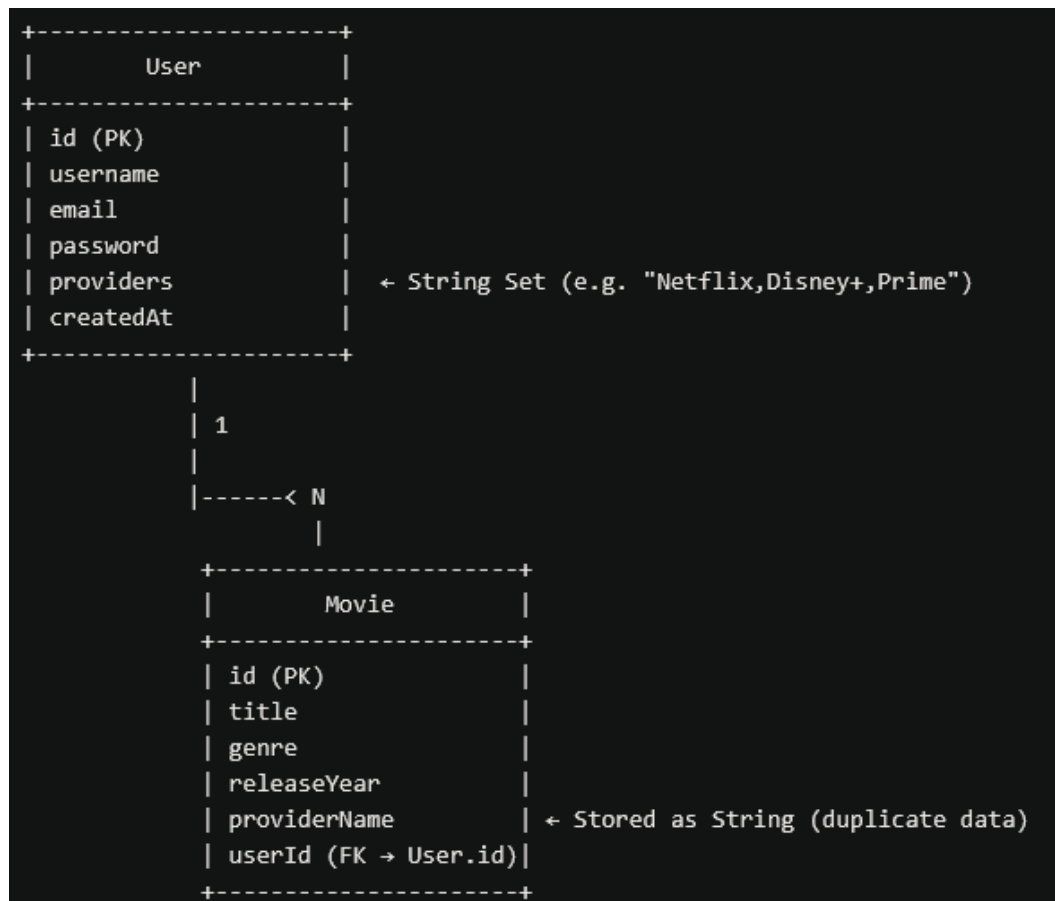


Abbildung 1: Erstes Entity-Relationship-Diagramm der Anwendung

die Bedeutung einer frühzeitigen Architekturentscheidung – insbesondere im Bereich der Sicherheits- und Authentifizierungsmechanismen.

2.6.2 Modellierung der Many-to-Many-Beziehung und strukturelle Weiterentwicklung

Im weiteren Verlauf der Systemarchitektur zeigte sich deutlich der Mehrwert einer sauberen relationalen Modellierung, insbesondere bei der Abbildung von Many-to-Many-Beziehungen.

Im Kontext der vorliegenden Applikation – einem Mediatheken-Abonnement-Manager – besteht zwischen den Entitäten *User* und *Provider* keine einfache 1:n-Beziehung, sondern eine n:m-Beziehung. Ein Benutzer kann mehrere Anbieter abonnieren, während ein Anbieter von mehreren Benutzern genutzt werden kann.

Die Einführung einer eigenständigen Relationstabelle (*UserProviderSubscription*) dient hierbei nicht ausschließlich der technischen Verknüpfung zweier Fremdschlüssel. Vielmehr übernimmt diese Tabelle eine fachlich zentrale Rolle innerhalb des Domä-



Abbildung 2: Überarbeitetes Entity-Relationship-Diagramm nach Integration von Keycloak

nenmodells. In der Relationstabelle werden zusätzliche Attribute gespeichert, die die konkrete Ausprägung der Beziehung beschreiben:

- Beginn des Abonnements (Startdatum)
- Ende des Abonnements (Enddatum)
- Zahlungszyklus (Enum: monatlich oder jährlich)

Diese Attribute gehören semantisch weder ausschließlich zum Benutzer noch ausschließlich zum Anbieter, sondern definieren die konkrete Vertragsbeziehung zwischen beiden Entitäten. Die Modellierung über eine separate Relationstabelle ermöglicht somit eine fachlich korrekte Abbildung der Geschäftslogik, die Erweiterbarkeit um weitere beziehungsbezogene Attribute – etwa Preis, Rabatt oder Kündigungsstatus – sowie eine klare Trennung der Verantwortlichkeiten innerhalb des Datenmodells. Gerade an dieser Stelle

wird deutlich, dass relationale Datenbanken ihre Stärke bei strukturierten und stark beziehungsorientierten Datenmodellen ausspielen.

2.6.3 Reflexion der anfänglichen Modellierungsentscheidung

Rückblickend stellt sich die anfängliche Entscheidung, Anbieter als `Set<String>` direkt innerhalb der User-Entität zu speichern, als struktureller Fehler heraus. Diese vereinfachte Modellierung war zwar für einen ersten Prototyp funktional ausreichend, führte jedoch zu mehreren konzeptionellen Einschränkungen:

1. Es konnten keine anbieterspezifischen Metadaten gespeichert werden (z. B. Website, Kategorie oder Beschreibung).
2. Die konkrete Ausgestaltung der Beziehung – Startdatum, Zahlungszyklus etc. – war nicht modellierbar.
3. Eine konsistente Datenhaltung war nicht gewährleistet, da String-Werte fehleranfällig und nicht referenziell abgesichert sind.
4. Erweiterungen des Systems hätten tiefgreifende strukturelle Änderungen erfordert.

Insbesondere der Umstand, dass beziehungsrelevante Informationen nicht abgebildet werden konnten, verdeutlicht die Problematik dieser frühen Designentscheidung. Die Speicherung als String-Set widersprach den Grundprinzipien der Normalisierung und verhinderte eine skalierbare Weiterentwicklung des Systems. Erst durch die Einführung einer eigenständigen **Provider**-Entität sowie einer Relationstabelle konnte das Datenmodell den fachlichen Anforderungen der Applikation gerecht werden. Diese Umstrukturierung stellt einen wesentlichen architektonischen Reifeprozess im Projektverlauf dar.

3 Architektur

3.1 Datenbanken

Um Dateneingaben persistent zu speichern und strukturiert zu verarbeiten, ist der Einsatz einer Datenbank unerlässlich. Eine Datenbank stellt die zentrale Instanz für die Speicherung, Organisation und Verwaltung von Daten dar. Sie ermöglicht nicht nur das einfache Ablegen von Informationen, sondern auch deren gezielte Abfrage, Aktualisierung und Löschung. Um auf die gespeicherten Daten zugreifen und diese verwalten zu können, wird ein sogenanntes Datenbankmanagementsystem (DBMS) eingesetzt, das als Vermittler zwischen der Anwendung und den physisch gespeicherten Daten fungiert.

3.1.1 Datenbank definition

Eine Datenbank ist eine strukturierte Sammlung von Daten, die so organisiert ist, dass sie effizient gespeichert, abgerufen und verwaltet werden kann. Im Gegensatz zur einfachen Dateiablage bietet eine Datenbank Mechanismen zur Vermeidung von Redundanzen, zur Sicherstellung der Datenkonsistenz sowie zur gleichzeitigen Nutzung durch mehrere Benutzer oder Anwendungen.

Grundlegende Begriffe im Datenbankumfeld sind:

- **Entität:** Ein eindeutig identifizierbares Objekt der realen Welt, das in der Datenbank abgebildet wird (z. B. ein Benutzer oder ein Streaming-Anbieter).
- **Attribut:** Eine Eigenschaft einer Entität (z. B. Name oder ID).
- **Relation:** Eine Tabelle in einer relationalen Datenbank, die Entitäten eines bestimmten Typs speichert.
- **Primärschlüssel:** Ein Attribut oder eine Kombination von Attributen, das eine Entität eindeutig identifiziert.

- **Fremdschlüssel:** Ein Attribut, das auf den Primärschlüssel einer anderen Tabelle verweist und damit Beziehungen zwischen Entitäten abbildet.

3.1.2 Datenbank Managment System - DBMS

Ein Datenbankmanagementsystem (DBMS) ist eine Software, die den Zugriff auf eine Datenbank verwaltet und kontrolliert. Es stellt die Schnittstelle zwischen der Anwendung und den physisch gespeicherten Daten dar und übernimmt Aufgaben wie die Speicherverwaltung, die Zugriffskontrolle, die Transaktionsverwaltung sowie die Sicherstellung der Datenkonsistenz.

Ein DBMS besteht im Wesentlichen aus zwei zentralen Komponenten. Der *physische Datenbestand* umfasst die tatsächlich auf einem Datenträger gespeicherten Daten. Der *Systemkatalog* – auch Data Dictionary genannt – enthält Metainformationen über die Struktur der Datenbank, also Informationen über Tabellen, Spalten, Datentypen, Beziehungen und Zugriffsrechte. Mithilfe des Systemkatalogs kann das DBMS Anfragen korrekt interpretieren und die gespeicherten Daten strukturiert zurückgeben.

Zu den bekanntesten DBMS zählen PostgreSQL, MySQL, Oracle Database und Microsoft SQL Server im relationalen Bereich sowie MongoDB und Redis im nicht-relationalen Bereich.

3.1.3 Datenbank Komponenten

Die Physische Speicherung aka der Datenbestand wo was gespeichert ist und der Systemkatalog mit dem ein DBMS dann auf die Daten zugreifen konnte.

3.2 Datenbankkonzept und architektonische Entscheidungen

Für das Backend der Anwendung spielt die Wahl der geeigneten Datenbanktechnologie eine zentrale Rolle. Trotz vorhandener Kenntnisse im Bereich nicht-relationaler Datenbanken (NoSQL) – wie etwa MongoDB oder Vektordatenbanken – wurde im Rahmen dieses Projekts bewusst eine relationale Datenbank eingesetzt.

Die Entscheidung für ein relationales Datenbanksystem basiert auf mehreren Faktoren. Die Anwendung weist eine klar strukturierte und logisch getrennte Datenarchitektur auf, wodurch sich die Daten und deren Beziehungen gut in einem relationalen Modell abbilden lassen. Entitäten und ihre Relationen konnten somit eindeutig definiert und konsistent strukturiert werden. Dieses Vorgehen entspricht dem etablierten Standard bei Anwendungen mit stark strukturierten Daten und klar definierten Geschäftsprozessen.

Ein weiterer wesentlicher Aspekt ist die Skalierbarkeit des Projekts. Relationale Datenbanken bieten durch Normalisierung, Transaktionssicherheit (ACID-Prinzip) und klar definierte Beziehungen eine stabile Grundlage für zukünftige Erweiterungen. Voraussetzung für eine nachhaltige Skalierbarkeit ist jedoch eine sorgfältig durchdachte Datenbankstruktur. Eine frühzeitig geplante und konsistente Modellierung – beispielsweise durch ein Entity-Relationship-Diagramm (ERD) – ist essenziell, um spätere strukturelle Anpassungen zu minimieren.

3.2.1 Herausforderungen im Entwicklungsprozess

Im Verlauf der Diplomarbeit traten mehrere strukturelle Herausforderungen auf, die im Sinne einer transparenten Dokumentation offen dargestellt werden sollen.

Zu Beginn der Implementierungsphase wurde ohne vollständig ausgearbeitetes Gesamtkonzept mit der Programmierung begonnen. Das Entity-Relationship-Diagramm wurde parallel zur Entwicklung erstellt, wodurch sich die Datenbankstruktur dynamisch entwickelte. Dieses iterative Vorgehen führte zwar zu schneller praktischer Umsetzung, hatte jedoch den Nachteil, dass strukturelle Änderungen im Nachhinein größeren Anpassungsaufwand verursachten.

Eine weitere wesentliche Herausforderung ergab sich durch die späte Integration von Keycloak als Identity- und Access-Management-Lösung. Ursprünglich war die Benutzerverwaltung eigenständig implementiert worden. Durch die nachträgliche Einführung von Keycloak mussten die bestehenden User-Strukturen grundlegend überarbeitet werden. Dies hatte zur Folge, dass die Benutzerentitäten angepasst werden mussten, die Authentifizierungs- und Autorisierungslogik neu strukturiert wurde sowie abhängige Klassen und Beziehungen refaktoriert und teilweise neu implementiert werden mussten. Diese Umstellung führte zu zusätzlichem Entwicklungsaufwand, zeigte jedoch zugleich

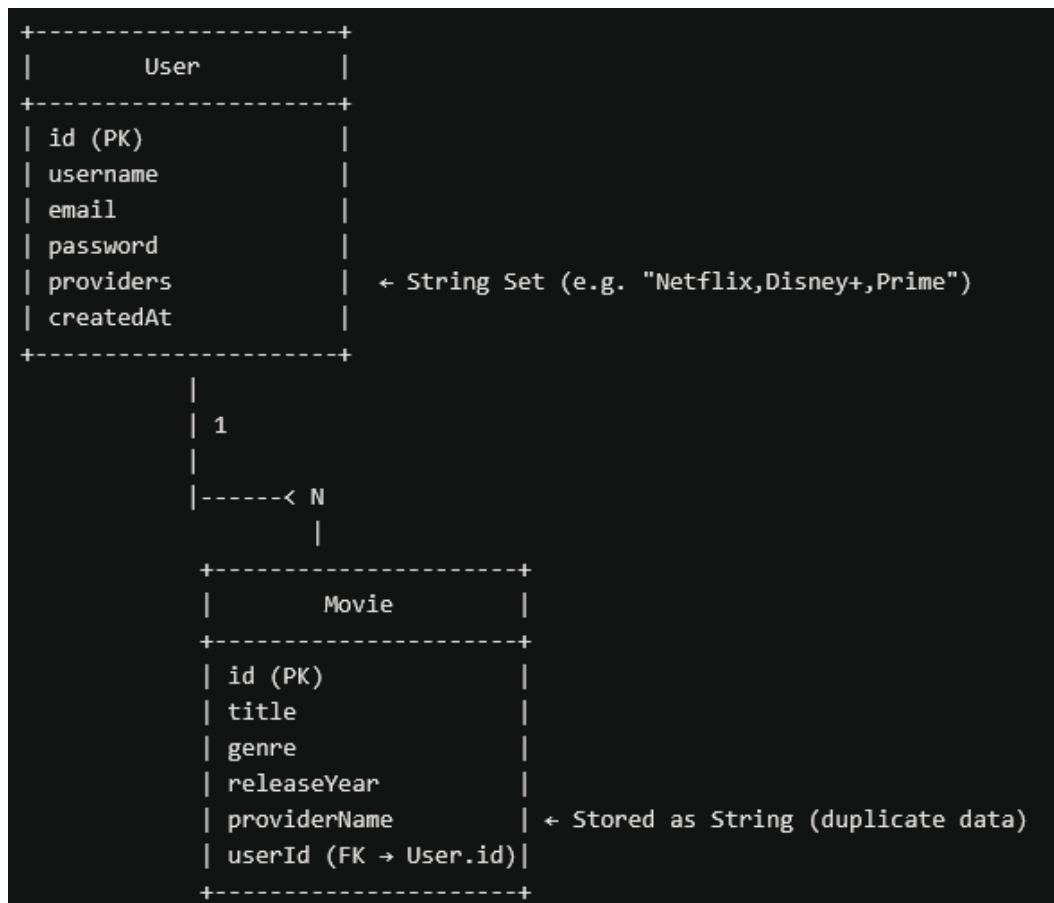


Abbildung 3: Erstes Entity-Relationship-Diagramm der Anwendung

die Bedeutung einer frühzeitigen Architekturentscheidung – insbesondere im Bereich der Sicherheits- und Authentifizierungsmechanismen.

3.2.2 Modellierung der Many-to-Many-Beziehung und strukturelle Weiterentwicklung

Im weiteren Verlauf der Systemarchitektur zeigte sich deutlich der Mehrwert einer sauberen relationalen Modellierung, insbesondere bei der Abbildung von Many-to-Many-Beziehungen.

Im Kontext der vorliegenden Applikation – einem Mediatheken-Abonnement-Manager – besteht zwischen den Entitäten *User* und *Provider* keine einfache 1:n-Beziehung, sondern eine n:m-Beziehung. Ein Benutzer kann mehrere Anbieter abonnieren, während ein Anbieter von mehreren Benutzern genutzt werden kann.

Die Einführung einer eigenständigen Relationstabelle (*UserProviderSubscription*) dient hierbei nicht ausschließlich der technischen Verknüpfung zweier Fremdschlüssel. Vielmehr übernimmt diese Tabelle eine fachlich zentrale Rolle innerhalb des Domä-



Abbildung 4: Überarbeitetes Entity-Relationship-Diagramm nach Integration von Keycloak

nenmodells. In der Relationstabelle werden zusätzliche Attribute gespeichert, die die konkrete Ausprägung der Beziehung beschreiben:

- Beginn des Abonnements (Startdatum)
- Ende des Abonnements (Enddatum)
- Zahlungszyklus (Enum: monatlich oder jährlich)

Diese Attribute gehören semantisch weder ausschließlich zum Benutzer noch ausschließlich zum Anbieter, sondern definieren die konkrete Vertragsbeziehung zwischen beiden Entitäten. Die Modellierung über eine separate Relationstabelle ermöglicht somit eine fachlich korrekte Abbildung der Geschäftslogik, die Erweiterbarkeit um weitere beziehungsbezogene Attribute – etwa Preis, Rabatt oder Kündigungsstatus – sowie eine klare Trennung der Verantwortlichkeiten innerhalb des Datenmodells. Gerade an dieser Stelle

wird deutlich, dass relationale Datenbanken ihre Stärke bei strukturierten und stark beziehungsorientierten Datenmodellen ausspielen.

3.2.3 Reflexion der anfänglichen Modellierungsentscheidung

Rückblickend stellt sich die anfängliche Entscheidung, Anbieter als `Set<String>` direkt innerhalb der User-Entität zu speichern, als struktureller Fehler heraus. Diese vereinfachte Modellierung war zwar für einen ersten Prototyp funktional ausreichend, führte jedoch zu mehreren konzeptionellen Einschränkungen:

1. Es konnten keine anbieterspezifischen Metadaten gespeichert werden (z. B. Website, Kategorie oder Beschreibung).
2. Die konkrete Ausgestaltung der Beziehung – Startdatum, Zahlungszyklus etc. – war nicht modellierbar.
3. Eine konsistente Datenhaltung war nicht gewährleistet, da String-Werte fehleranfällig und nicht referenziell abgesichert sind.
4. Erweiterungen des Systems hätten tiefgreifende strukturelle Änderungen erfordert.

Insbesondere der Umstand, dass beziehungsrelevante Informationen nicht abgebildet werden konnten, verdeutlicht die Problematik dieser frühen Designentscheidung. Die Speicherung als String-Set widersprach den Grundprinzipien der Normalisierung und verhinderte eine skalierbare Weiterentwicklung des Systems. Erst durch die Einführung einer eigenständigen **Provider**-Entität sowie einer Relationstabelle konnte das Datenmodell den fachlichen Anforderungen der Applikation gerecht werden. Diese Umstrukturierung stellt einen wesentlichen architektonischen Reifeprozess im Projektverlauf dar.

4 Datenbank

4.1 Datenmodell für Abonnements und Medien

4.2 Begründung für relationale Datenhaltung

4.2.1 Einsatz von JPA und ORM

Nicht mehr im Inhaltsverzeichnis.

Deepest

Vermeide mich.

5 Umsetzung

Siehe tolle Daten in Tab. 1.

Siehe und staune in Abb. 5. Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetur at, consectetur sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus. Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae, dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui.

	Regular Customers	Random Customers
Age	20-40	>60
Education	university	high school

Tabelle 1: Ein paar tabellarische Daten

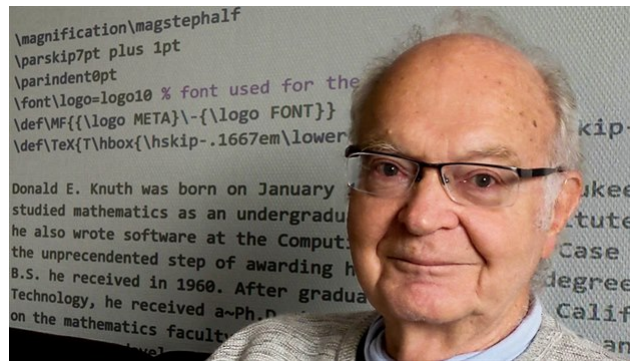


Abbildung 5: Don Knuth – CS Allfather

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetur a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetur. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus. Dann betrachte den Code in Listing 4.

Listing 4: Some code

```

1  # Program to find the sum of all numbers stored in a list
   (the not-Pythonic-way)
2
3  # List of numbers
4  numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]
5
6  # variable to store the sum
7  sum = 0
8
9  # iterate over the list
10 for val in numbers:
11     sum = sum+val
12
13 print("The sum is", sum)

```

6 Zusammenfassung

Aufzählungen:

- Itemize Level 1
 - Itemize Level 2
 - Itemize Level 3 (vermeiden)
- 1. Enumerate Level 1
 - a. Enumerate Level 2
 - i. Enumerate Level 3 (vermeiden)

Desc Level 1

Desc Level 2 (vermeiden)

Desc Level 3 (vermeiden)

Glossar

GUID Globally Unique Identifier

Literaturverzeichnis

Abbildungsverzeichnis

1	Erstes Entity-Relationship-Diagramm der Anwendung	14
2	Überarbeitetes Entity-Relationship-Diagramm nach Integration von Key-cloak	15
3	Erstes Entity-Relationship-Diagramm der Anwendung	20
4	Überarbeitetes Entity-Relationship-Diagramm nach Integration von Key-cloak	21
5	Don Knuth – CS Allfather	25

Tabellenverzeichnis

1	Ein paar tabellarische Daten	24
---	--	----

Quellcodeverzeichnis

1	Auszug einer TMDB API-Antwort für einen Film	6
2	TrendingMovieDTO als Java Record	7
3	Zusammensetzung der Poster-URL im TmdbService	7
4	Some code	25

Anhang